

System Design Document (SDD): AI Investment Research Bot

1. Introduction **Purpose:** This document defines the architecture, components, data design, and workflows for the AI Investment Research Bot. The system automates ingestion, analysis, and summarization of financial market data to produce research insights. **Scope:** The system ingests structured and unstructured data (financial statements, SEC filings, prices, and news), processes them through analytics and AI pipelines, and enables analysts to query the data via a chat interface.

2. System Overview The AI Investment Research Bot consists of multiple interconnected modules: Data Ingestion Layer: Fetches data from EDGAR, news APIs, and price feeds. Storage Layer: PostgreSQL (structured data), Neo4j (graph relationships), and a vector database (embeddings). Analytics Layer: Performs financial computations, time-series forecasting, and entity linking. AI/NLP Layer: Uses RAG (Retrieval-Augmented Generation) to answer analyst queries with contextual data. Frontend/API Layer: FastAPI backend with React-based chat UI and dashboards.

3. Architecture The architecture follows a modular microservices-based design with orchestration through Airflow or Prefect. Data pipelines populate the relational and graph databases, while AI models consume these data for analysis. **Core Components:** Data Ingestion: Python ETL, APIs, Scrapers Databases: PostgreSQL, Neo4j, Vector DB Model Layer: Financial metrics engine, LLM (OpenAI or local), Embeddings generator Backend: FastAPI, Celery Frontend: React (Chat UI, visualizations) Infra: Docker, Kubernetes, AWS/GCP

4. Data Model **PostgreSQL Entities:** companies(company_id, name, sector, cik, description) tickers(ticker_id, company_id, ticker, exchange) daily_prices(price_id, company_id, date, open, close, volume) financial_statements(fs_id, company_id, statement_type, line_item, value, period_end) filings(filing_id, company_id, form_type, filing_date, url, content) news_articles(article_id, company_id, published_at, title, content, sentiment) **Neo4j Relationships:** (Company)-[:DEPENDS_ON]->(Company) (Institution)-[:OWNS]->(Company) (Document)-[:MENTIONS]->(Company) (Company)-[:PARTNER_WITH]->(Company)

5. Data Flow 1. Data is fetched from sources (EDGAR, APIs, RSS) via the Ingestion Layer. 2. Raw data is stored in S3 and processed into normalized tables in PostgreSQL. 3. Entities and relationships are extracted and stored in Neo4j. 4. Text documents are embedded into a Vector DB for retrieval. 5. User queries trigger RAG-based responses combining structured and unstructured data.

6. API Design **Core Endpoints (FastAPI):** POST /v1/query — Execute research query (RAG) GET /v1/company/{ticker}/summary — Retrieve company analysis POST /v1/ingest/filing — Trigger filing ingestion GET /v1/portfolio/impact — Scenario impact on portfolio POST /v1/alerts — Manage user alerts

7. Security and Compliance **Data Security:** All data encrypted at rest and in transit. Role-based access control via JWT/OAuth2. **Compliance:** Ensure licensing for market data sources. Provide disclaimers for non-advisory use. Maintain audit logs and prompt/response traceability.

8. Non-Functional Requirements **Scalability:** Horizontally scalable ingestion and query services. **Availability:** Target 99.5% uptime with redundancy. **Performance:** Query response within 6 seconds. **Factual Accuracy:** 95%+ accuracy for numeric data from filings. **Observability:** Centralized logging, metrics (Grafana), and alerts.

9. Implementation Roadmap (8 Weeks) **Week 1–2:** Setup infra, databases, and basic ETL for prices & filings. **Week 3–4:** Embed documents, enable RAG-based query engine. **Week 5:** Build analytics engine for financial ratios. **Week 6:** Develop API endpoints and frontend chat UI. **Week 7:**

Implement alerts, testing, and monitoring. **Week 8:** Final integration, documentation, and deployment.